AutoShip — Automated CI/CD Pipeline with Jenkins, Ansible & AWS S3 (DevOps Project)

Overview

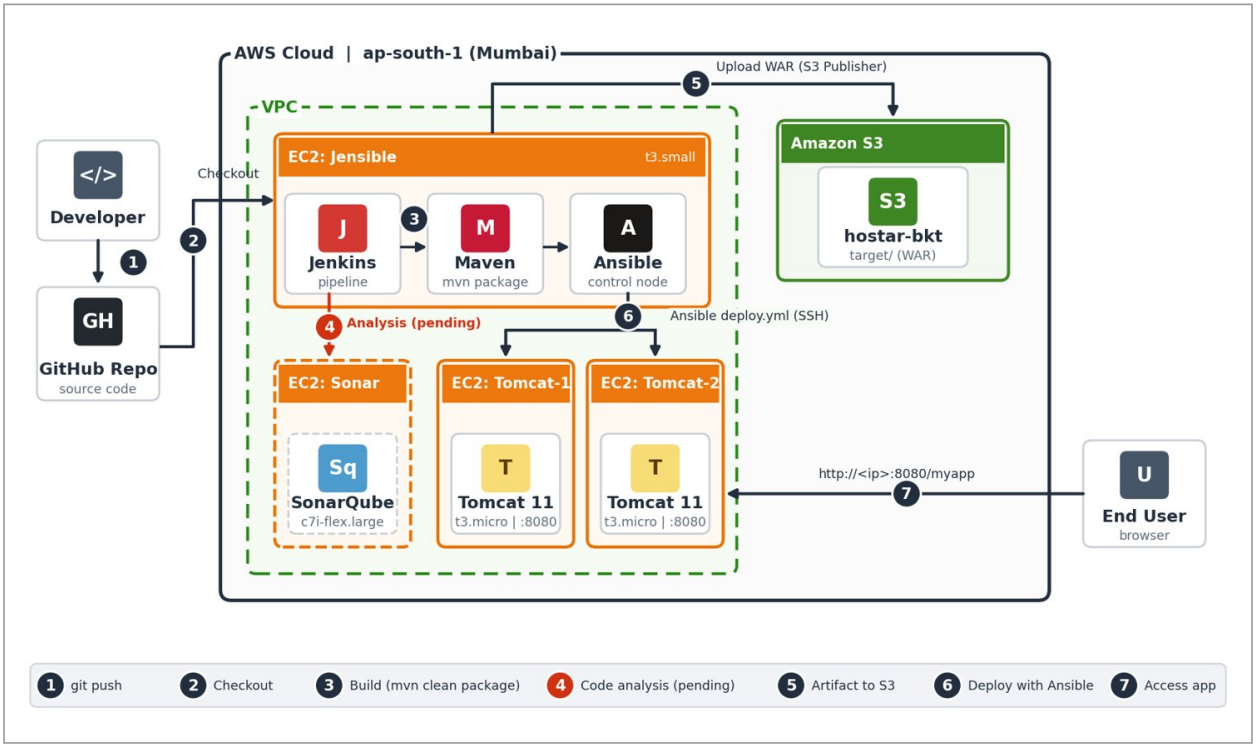
AutoShip is an end-to-end **CI/CD pipeline on AWS**, built as a hands-on DevOps project to demonstrate practical skills in Linux, automation, configuration management, build tooling, artifact storage and application deployment.

A single Jenkins pipeline pulls code from Git, builds it with **Maven**, stores the build artifact (`.war`) in **Amazon S3**, and uses **Ansible** to deploy it automatically to two **Apache Tomcat** servers. A **SonarQube** server is installed for code-quality analysis; its integration with Jenkins is the one pending item (see Known Limitations).

Project Status: Checkout, Build, Artifact upload and Deploy stages are working end to end. SonarQube integration is pending.

Architecture Diagram

The diagram below shows the complete flow — source code from GitHub, build and orchestration on the Jenkins/Ansible server, artifacts to S3, and automatic deployment to two Tomcat servers. The dashed red line marks the SonarQube integration that is still pending.



AutoShip Architecture: CI/CD flow from GitHub to Tomcat on AWS (steps 1 to 7)

High-level flow:

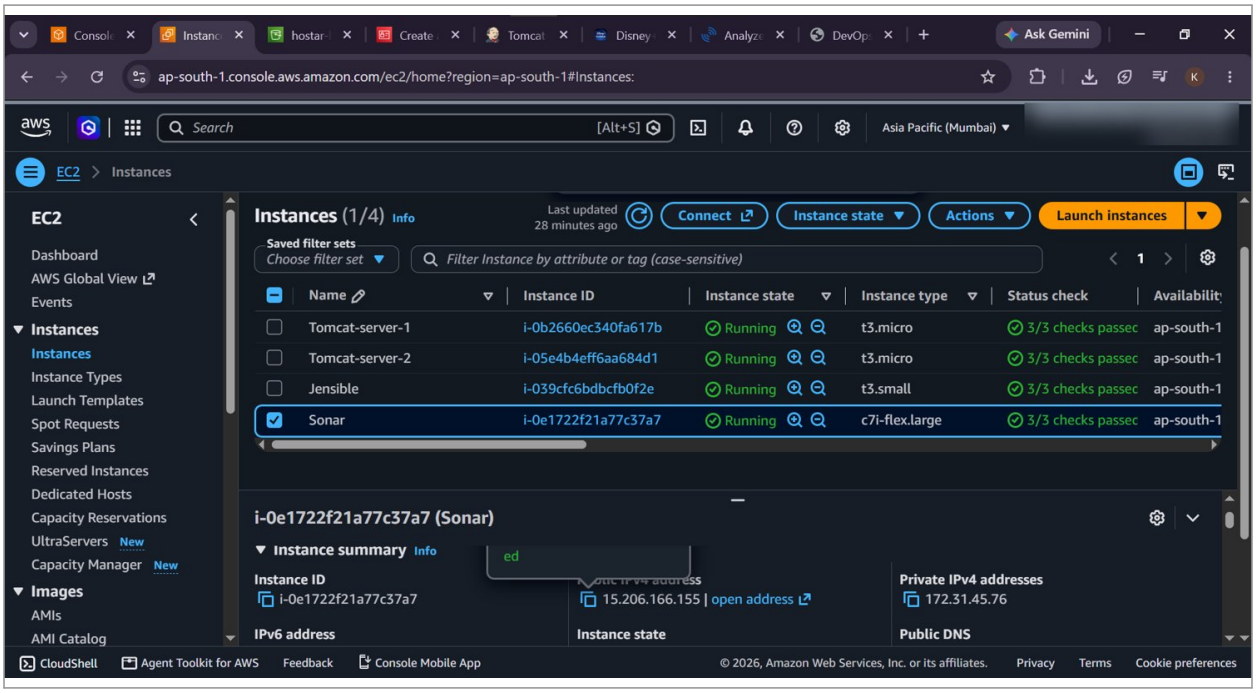
```
Developer -> GitHub -> Jenkins (Checkout)
    -> Maven build (myapp.war)
        -> Test stage (SonarQube planned)
            -> S3 bucket hostar-bkt (artifact)
                -> Ansible deploy.yml
                    -> Tomcat-server-1 and Tomcat-server-2 (:8080/myapp)
```

---

1. Infrastructure (EC2)

Server	Instance Type	Role
Jensible	t3.small	Jenkins server and Ansible control node
Tomcat-server-1	t3.micro	Ansible managed node, Tomcat application server
Tomcat-server-2	t3.micro	Ansible managed node, Tomcat application server
Sonar	c7i-flex.large	SonarQube server (port 9000)

Component	Details
Region	ap-south-1 (Mumbai)
Operating System	Amazon Linux 2023 on all four servers
Ports used	22 (SSH), 8080 (Jenkins and Tomcat), 9000 (SonarQube)
Connectivity	Control node reaches managed nodes over private IPs (same VPC)



EC2 console: all four servers running with 3/3 status checks passed.

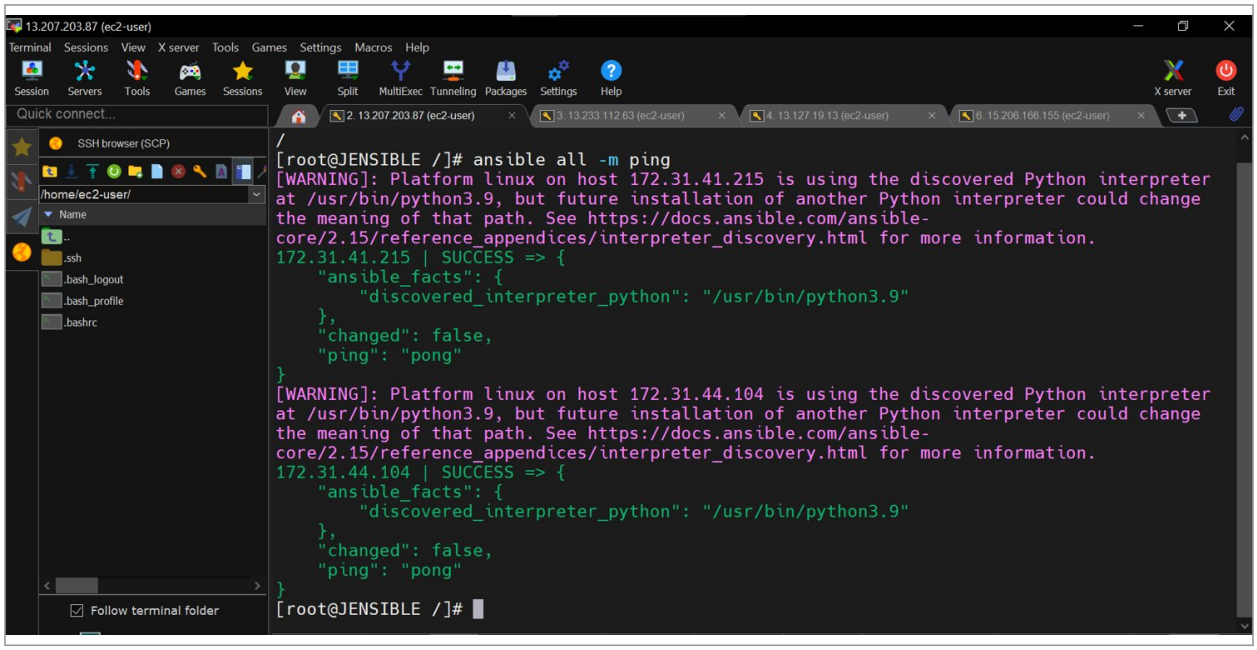
---

2. Ansible Setup (Control Node and Managed Nodes)

Passwordless SSH was configured from the control node (Jensible) to both Tomcat servers so Ansible and Jenkins can run playbooks without prompts.

- Set a password on all three machines and enabled root login in `/etc/ssh/sshd_config`, then restarted `sshd`.
- Installed Python 3, pip and Ansible on the control node.
- Added the managed nodes (private IPs) to `/etc/ansible/hosts`.
- Generated an SSH key (`ssh-keygen`) and copied it to each managed node with `ssh-copy-id root@<private-ip>`.
- Verified connectivity with `ansible all -m ping`.

```
yum update -y
yum install python3 python3-pip ansible -y
vi /etc/ansible/hosts
ssh-keygen
ssh-copy-id root@<private-ip>
ansible all -m ping
```



Ansible ping returned SUCCESS ("ping": "pong") for both managed nodes.

---

3. Jenkins Setup

Jenkins was installed on the control node using an install script. The following plugins were installed and Jenkins was restarted:

Plugin	Purpose
Deploy to container	Deploy WAR files to application containers
S3 Publisher	Upload build artifacts to Amazon S3
Ansible	Run Ansible playbooks from pipeline stages

Under **Manage Jenkins → Tools → Ansible**, an installation named `ansible` was added with path `/bin`. Credentials for the managed nodes (Linux username and password) were added in **Manage Jenkins → Credentials** so the Ansible step can connect to the Tomcat servers.

---

4. Tomcat Installation with Ansible

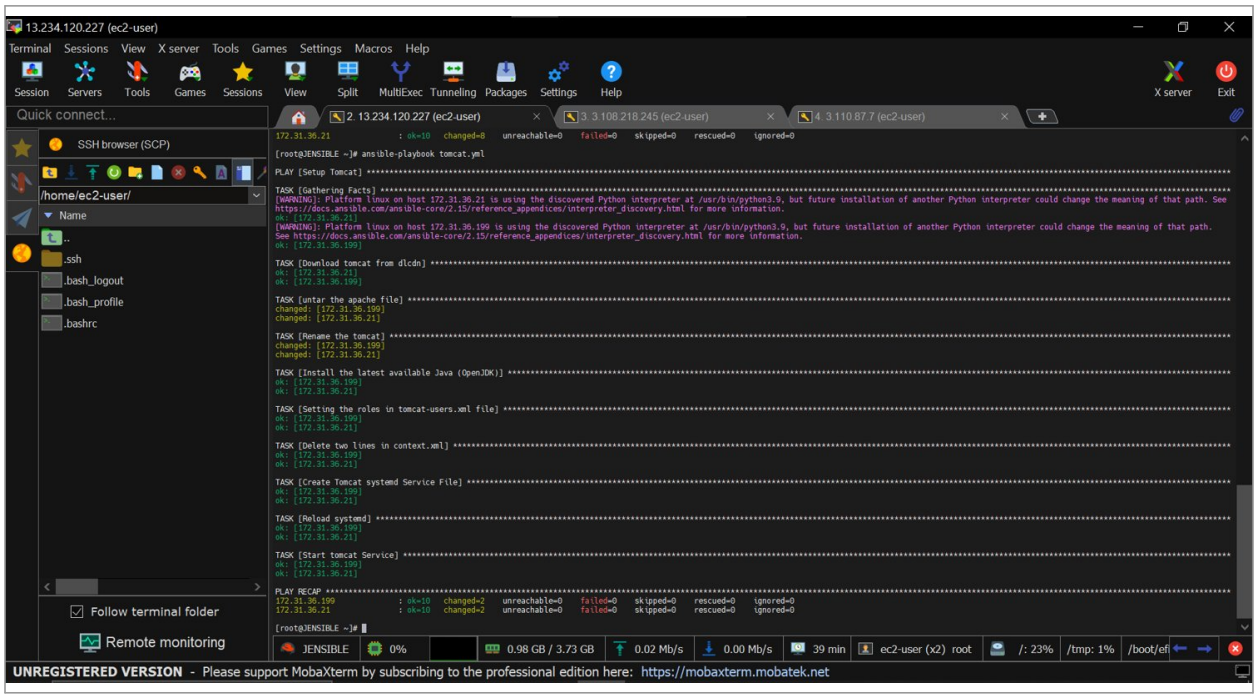
Tomcat 11.0.26 was installed on both managed nodes using one playbook (`tomcat.yml`). It downloads Tomcat, installs Java 17 (Amazon Corretto), replaces two config files (`tomcat-users.xml` and `context.xml`, kept in `/root/` of the control node) so the Manager app is reachable, creates a `systemd` service and starts Tomcat.

```
---
- name: Setup Tomcat
  hosts: all
  become: yes
  tasks:
    - name: Download tomcat from dlcdn
      get_url:
        url: "https://dlcdn.apache.org/tomcat/tomcat-11/v11.0.26/bin/apache-tomcat-11.0.26.tar.gz"
        dest: "/root/"
    - name: untar the apache file
      command: tar -zxvf apache-tomcat-11.0.26.tar.gz
    - name: Rename the tomcat
      command: mv apache-tomcat-11.0.26 tomcat
    - name: Install the latest available Java (OpenJDK)
      yum:
        name: java-17-amazon-corretto
        state: present
    - name: Setting the roles in tomcat-users.xml file
      template:
        src: tomcat-users.xml
        dest: /root/tomcat/conf/tomcat-users.xml
    - name: Delete two lines in context.xml
      template:
        src: context.xml
        dest: /root/tomcat/webapps/manager/META-INF/context.xml
    - name: Create Tomcat systemd Service File
      copy:
        dest: /etc/systemd/system/tomcat.service
        content: |
          [Unit]
          Description=Apache Tomcat Server
          After=network.target
          [Service]
          User=root
          Group=root
          Type=forking
          Environment="JAVA_HOME=/usr/lib/jvm/jre"
          Environment="CATALINA_HOME=/root/tomcat"
```

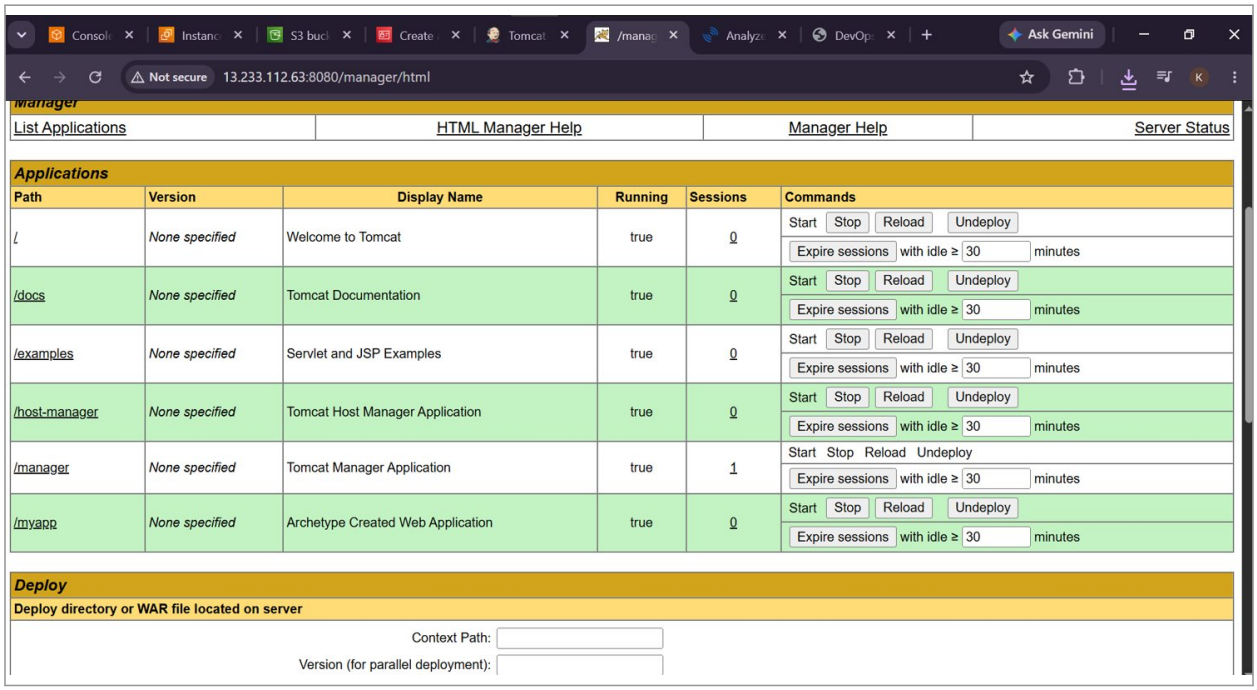
```
        ExecStart=/root/tomcat/bin/startup.sh
        ExecStop=/root/tomcat/bin/shutdown.sh
        Restart=on-failure
        [Install]
        WantedBy=multi-user.target
    - name: Reload systemd
      systemd:
        daemon_reload: yes
    - name: Start tomcat Service
      service:
        name: tomcat
        state: started
        enabled: yes
```

The playbook was validated and executed with:

```
ansible-playbook --syntax-check tomcat.yml
ansible-playbook tomcat.yml
```



Playbook run completed on both nodes: failed=0, unreachable=0 in the PLAY RECAP.



Tomcat Manager page confirms Tomcat is running and the custom configuration was applied.

---

5. Jenkins Pipeline

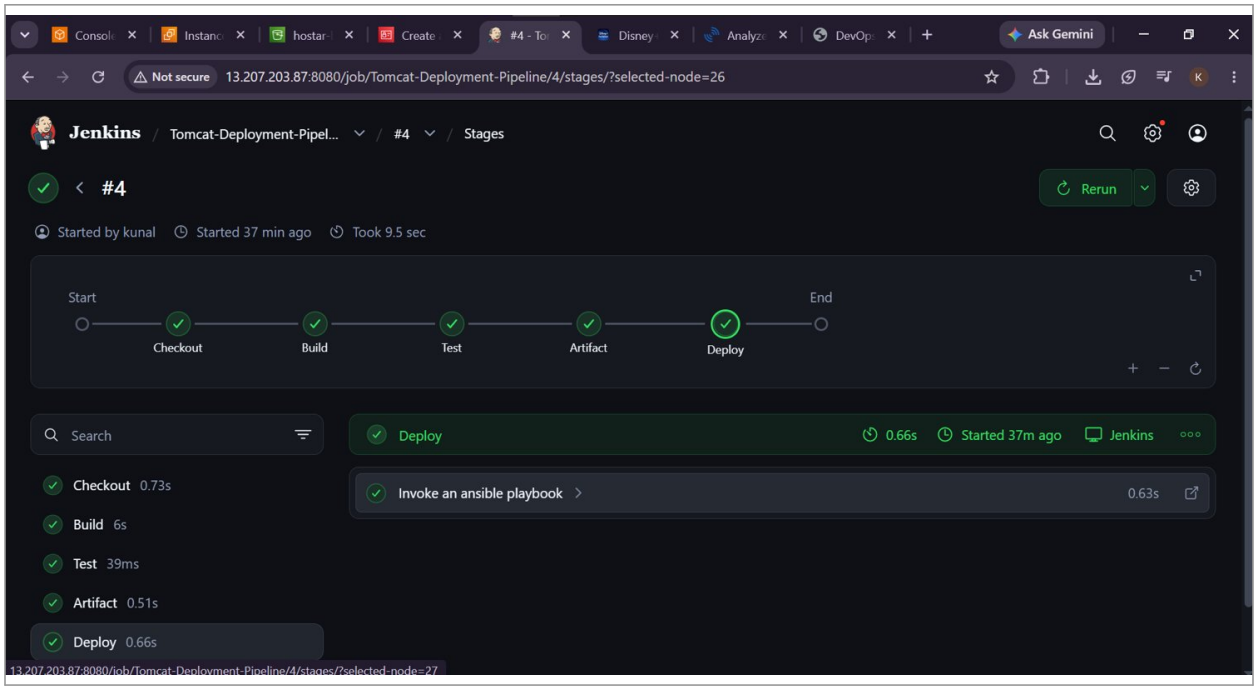
Job name: `Tomcat-Deployment-Pipeline`. The pipeline has five stages:

Stage	What it does
Checkout	Pulls the source code from GitHub
Build	Runs `mvn clean package` to generate `myapp.war`
Test	Placeholder stage. SonarQube analysis will be added here (pending)
Artifact	Uploads the WAR to S3 bucket `hostar-bkt` using S3 Publisher
Deploy	Invokes the Ansible playbook `deploy.yml` to push the WAR to both Tomcat servers

```
pipeline {
    agent any
    stages {
        stage('Checkout') { steps { git '<github-repo-url>' } }
        stage('Build')    { steps { sh 'mvn clean package' } }
        stage('Test')     { steps { echo 'SonarQube analysis planned here' } }
        stage('Artifact') {
            steps {
                s3Upload(bucket: 'hostar-bkt', selectedRegion: 'ap-south-1',
                         sourceFile: '**/*.war', profileName: 's3-profile-creds', ...)
            }
        }
        stage('Deploy') {
            steps {
                ansiblePlaybook(installation: 'ansible',
                                playbook: '/etc/ansible/deploy.yml',
                                inventory: '/etc/ansible/hosts',
                                credentialsId: '<linux-credentials-id>',
                                disableHostKeyChecking: true)
```

```
            }
        }
    }
}
```

Pipeline shown in simplified form; the S3 and Ansible steps were generated using Jenkins' Pipeline Syntax snippet generator.

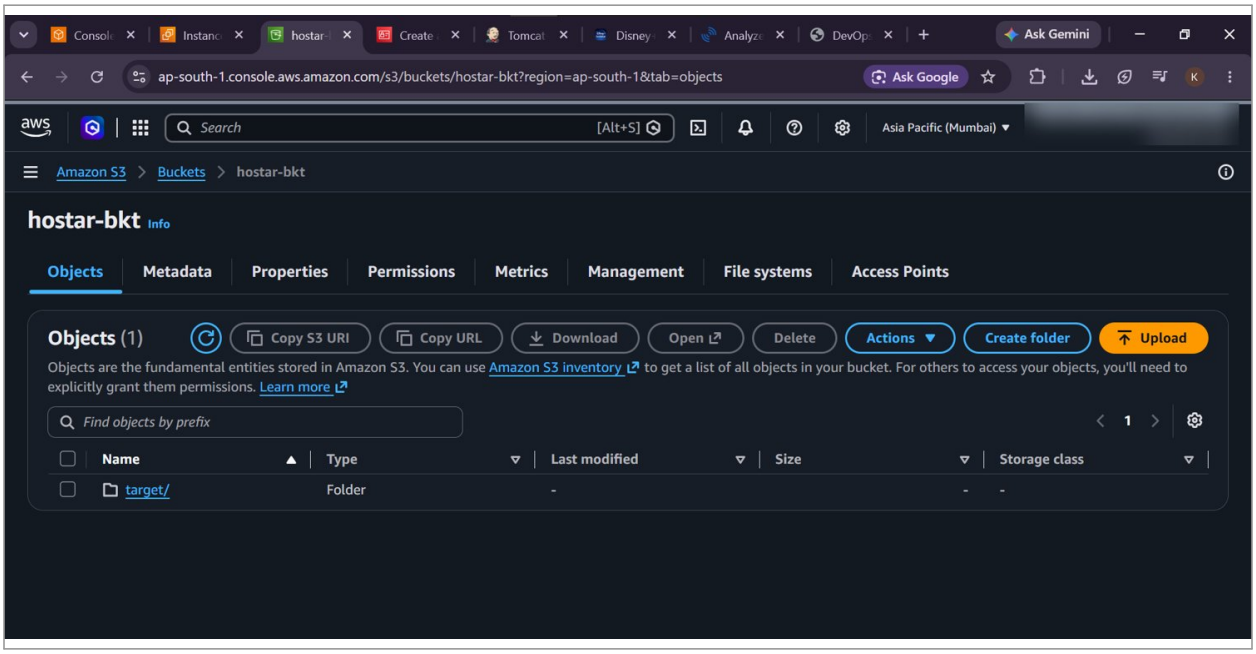


Build #4 of Tomcat-Deployment-Pipeline: all five stages successful (total 9.5 sec).

---

6. Artifact Storage (Amazon S3)

Attribute	Value
Bucket	hostar-bkt
Region	ap-south-1
Content	WAR artifact uploaded under target/ by the pipeline
Jenkins profile	s3-profile-creds (added in S3 Publisher settings)



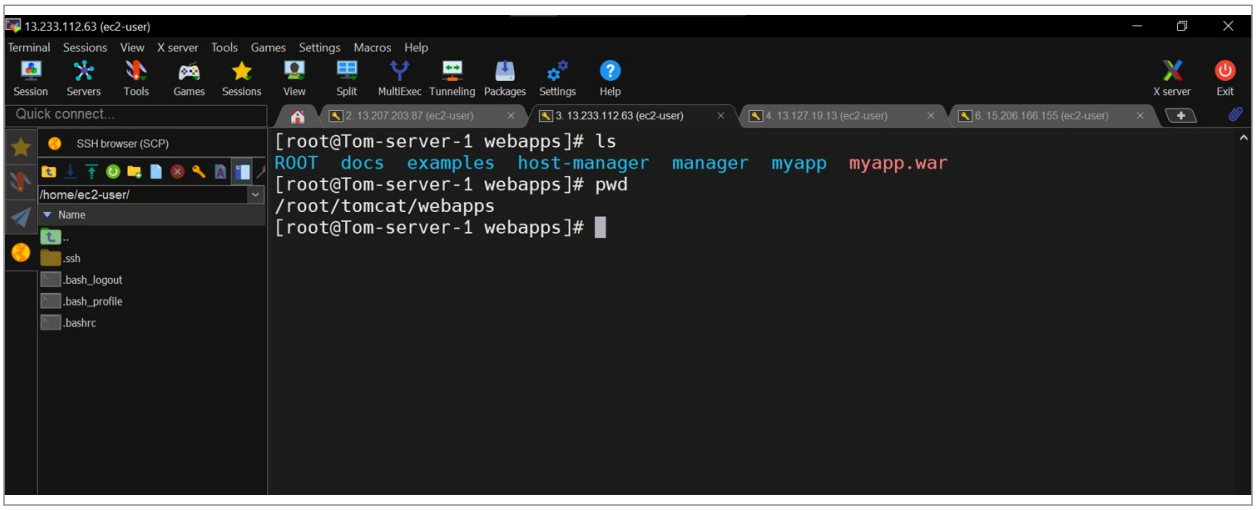
S3 bucket hostar-bkt with the target/ folder created by the Artifact stage.

---

7. Automated Deployment (deploy.yml)

A second playbook, `/etc/ansible/deploy.yml`, copies the WAR generated by the Build stage into Tomcat's `webapps` directory on both servers. It is not run manually; Jenkins invokes it in the Deploy stage. Tomcat auto-extracts the WAR into the `myapp` application.

```
---
- name: Deploy war to tomcat servers
  hosts: all
  tasks:
    - name: Give a war file to ansible
      copy:
        src: /var/lib/jenkins/workspace/ansible-pipeline/target/myapp.war
        dest: /root/tomcat/webapps
```



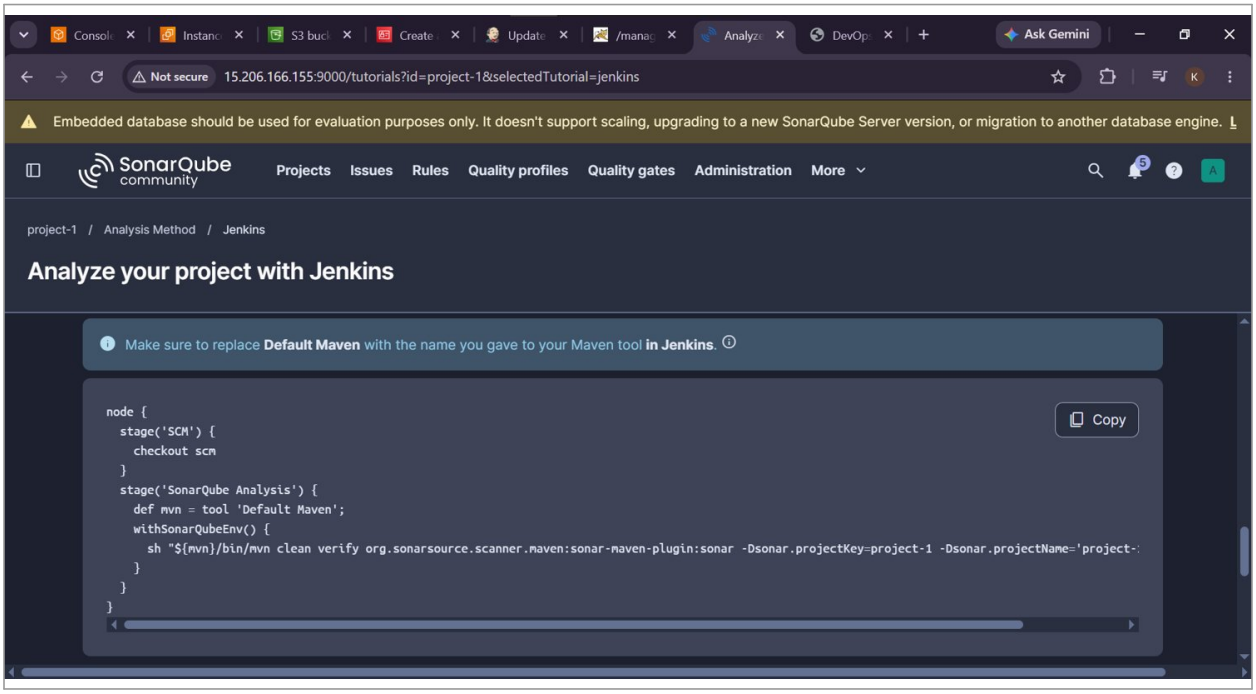
Tomcat-server-1: myapp.war and the extracted myapp folder inside /root/tomcat/webapps.

---

8. Code Quality (SonarQube)

SonarQube was installed on a dedicated server (c7i-flex.large, Amazon Linux 2023, with an IAM role attached) using the script below, and started manually. The server is reachable on port 9000 and a project was created in the UI.

```
#!/bin/bash
cd /opt/
wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-26.9.0.129388.zip
unzip sonarqube-26.9.0.129388.zip
sudo dnf install java-21-amazon-corretto -y
useradd sonar
chown sonar:sonar sonarqube-26.9.0.129388 -R
su - sonar
# start manually as the sonar user:
# sh /opt/sonarqube-26.9.0.129388/bin/linux-x86-64/sonar.sh start
```



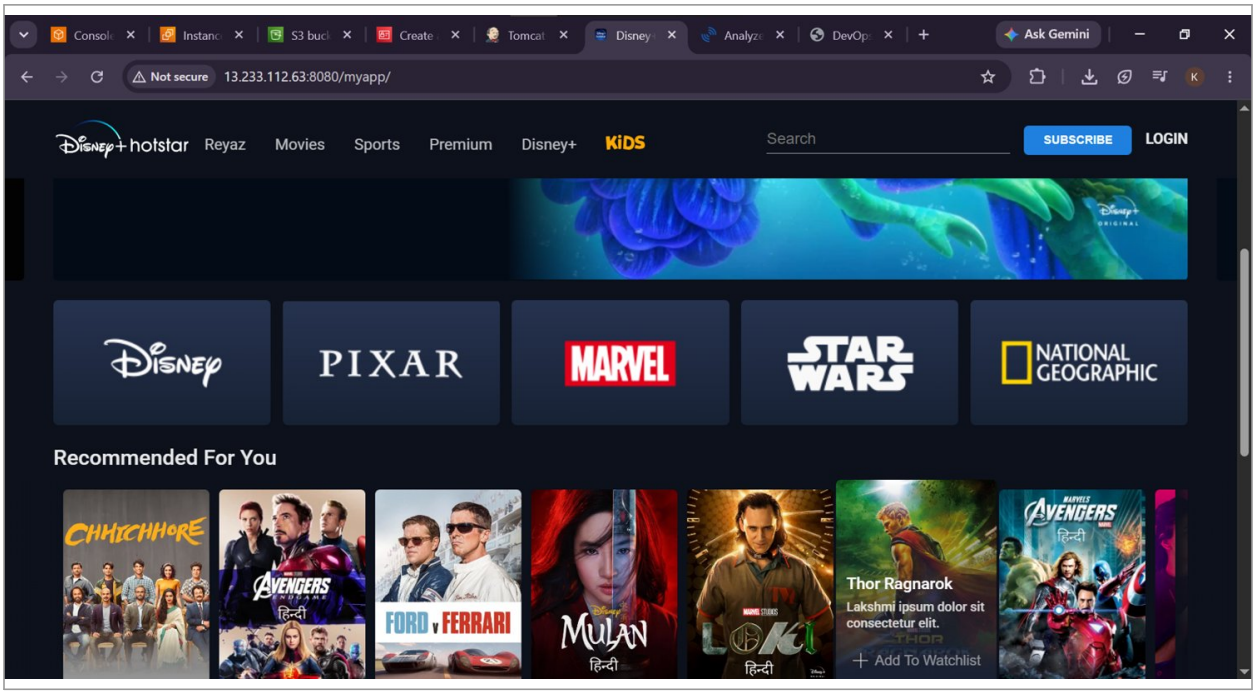
SonarQube Community running on port 9000. This is SonarQube's own "Analyze your project with Jenkins" guide, the next step to complete.

---

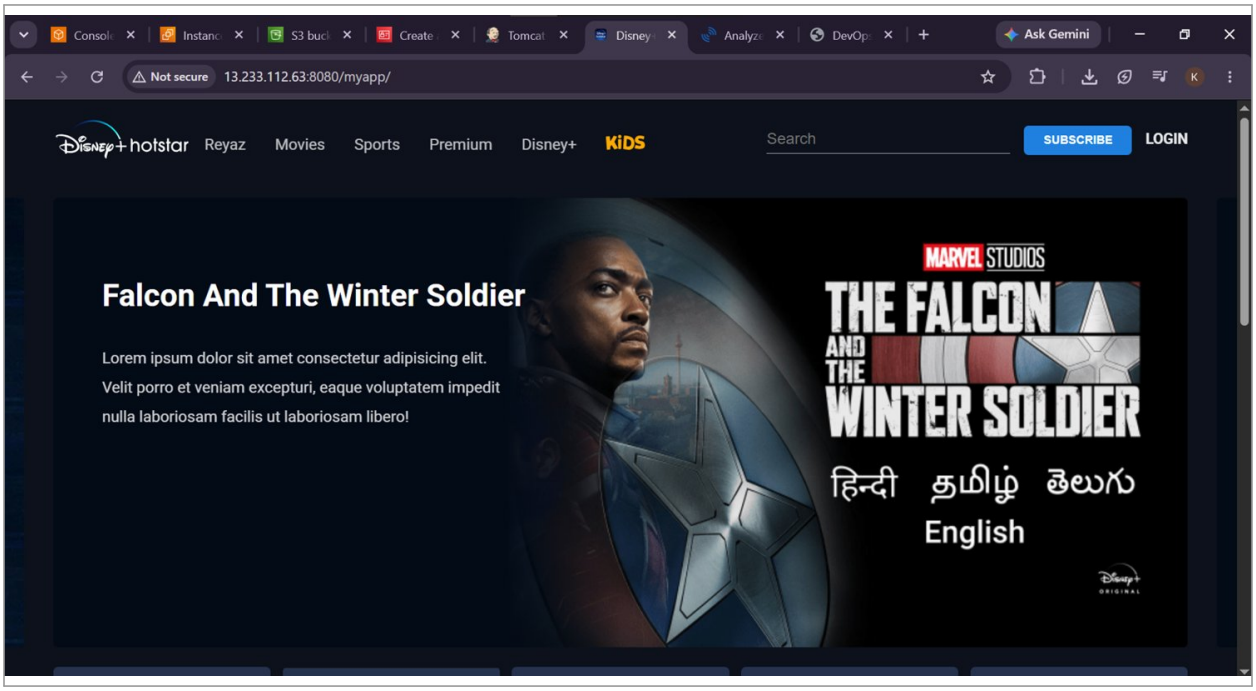
9. Testing & Verification

- Verified connectivity from the control node to both managed nodes with `ansible all -m ping` (SUCCESS on both).
- Verified Tomcat installation: playbook recap showed `failed=0` and `unreachable=0` on both servers; Tomcat Manager page opened in the browser.
- Verified the pipeline: build #4 completed all five stages (Checkout, Build, Test, Artifact, Deploy) successfully.
- Verified the artifact in S3: `target/` folder created in `hostar-bkt`.
- Verified deployment: `myapp.war` present in Tomcat `webapps` and the application running (`/myapp` shown as running in Tomcat Manager).

- Verified end to end by opening `http://<tomcat-ip>:8080/myapp/` in a browser; the sample web application loaded.



Final output 1: deployed sample web application served from Tomcat at :8080/myapp/.



Final output 2: the same application after scrolling, confirming all static assets were served.

---

10. Known Limitations

Item	Status	Reason / Next step
SonarQube to Jenkins integration	Pending (server installed and running)	Test stage is currently a placeholder. Remaining: generate a SonarQube token, install the SonarQube Scanner plugin in Jenkins, add the server URL and token under Manage Jenkins → System, add the Maven tool, and replace the Test stage with a `withSonarQubeEnv()` stage running the sonar-maven-plugin.
Quality Gate check	Not started	To be added after the integration so the pipeline fails on poor code quality.
Automatic trigger	Not started	Add a GitHub webhook so a push starts the pipeline automatically.

The SonarQube integration is documented honestly as the remaining step rather than hidden: the server, project and Jenkins pipeline are all ready, and the architecture is designed so the analysis plugs into the existing **Test** stage.

Security notes (learning environment): root SSH login and password authentication were enabled to set up Ansible, Tomcat runs as root, and SonarQube was installed with default settings. For production these would be replaced with key-based auth on a non-root user, a dedicated Tomcat user, restricted Security Group sources and a changed SonarQube admin password.

---

11. Tech Stack Summary

- **Cloud:** AWS EC2, S3, IAM (ap-south-1)
- **Operating System:** Amazon Linux 2023
- **Source Control:** Git, GitHub
- **Build:** Apache Maven
- **CI/CD:** Jenkins (declarative pipeline)
- **Configuration Management and Deployment:** Ansible (playbooks, inventory, SSH keys)
- **Application Server:** Apache Tomcat 11.0.26 on Java 17 (Amazon Corretto)
- **Artifact Storage:** Amazon S3 (S3 Publisher plugin)
- **Code Quality:** SonarQube Community (Java 21), integration pending

Project Status

End-to-end functional — code is checked out, built, stored in S3 and deployed automatically to two Tomcat servers through a single Jenkins pipeline. SonarQube is installed and running; its Jenkins integration is the one intentionally pending component and the next step on the roadmap.

Author: Kunal Karale | GitHub: github.com/Kunal-Karale | LinkedIn: linkedin.com/in/kunal-karale